Asistente RAG reproducible sobre prensa y Bluesky colombianos con Milvus y LLM locales

Abel Albuez Sánchez, Kelly Joane León Torres, Juan Camilo Torres Peña y Jesús David Romero Melo
Maestría en Inteligencia Artificial, Pontificia Universidad Javeriana
Profesor: Ing. Luis Gabriel Moreno Sandoval, PhD
Repositorio: https://github.com/AbelAlbuez/natural-language-processing-workshops

***Resumen*—Este documento describe el corpus, la segmentación, la arquitectura y el protocolo de evaluación de un asistente de generación aumentada por recuperación (RAG) sobre noticias y publicaciones de Bluesky en español e inglés. El corpus preparado contiene 146.923 documentos y la segmentación por tokens 256/48 produce 577.324 chunks, ambos verificados. La pila prevista usa Milvus standalone v2.6.25 sin MinIO, ya validado, y modelos de lenguaje locales servidos con Ollama. Los pesos del encoder `intfloat/multilingual-e5-small` están descargados y cargan sin conexión. El protocolo divide un gold standard de 40 preguntas en 20 de desarrollo y 20 de prueba. Los embeddings de los 577.324 chunks están calculados y cargados en Milvus con una ingesta por lotes idempotente, y la API responde con citas a los fragmentos recuperados. La configuración de generación quedó congelada con temperatura 0 y `repeat_penalty` 1,1. El gold standard, la calibración del umbral de abstención, los experimentos y la evaluación humana están pendientes; este documento no presenta resultados experimentales. Las referencias bibliográficas están por verificar.**

***Index Terms*—RAG, recuperación de información, corpus bilingüe, segmentación, embeddings, Milvus, LLM locales, evaluación**

I. Problema e hipótesis

El proyecto estudia la recuperación de evidencia en un corpus de prensa y publicaciones sociales colombianas en español e inglés. La hipótesis de trabajo PI6 propone que Recall@5 es menor para Bluesky que para Media Cloud al comparar preguntas del mismo idioma por separado, y que la abstención correcta es menor para preguntas en inglés que para preguntas en español. Es una hipótesis pendiente de evaluación.

El protocolo prevé reportar Recall@k ($k \in \{1, 3, 5, 10\}$), MRR, nDCG@10, abstención correcta y fidelidad de citas, desglosados por fuente esperada e idioma. Las métricas siguen referencias candidatas aún no verificadas [24], [27].

II. Corpus y perfilado (A)

El diagnóstico del proyecto registra 169.248 documentos originales y 146.923 después de la preparación bilingüe: 64.666 en español y 82.257 en inglés. El recuento de 146.923 filas del Parquet preparado se comprobó por metadatos. Las cifras se documentan en `docs/diagnostico_corpus.md` y `docs/prepare_stats_es_en.json`; no se incluye texto de documentos. La Figura 1 desglosa el corpus preparado por fuente e idioma.

Cuadro I
Tamaño del corpus preparado.

Etapa o idioma	Documentos
Originales	169.248
Preparados, total	146.923
Español	64.666
Inglés	82.257

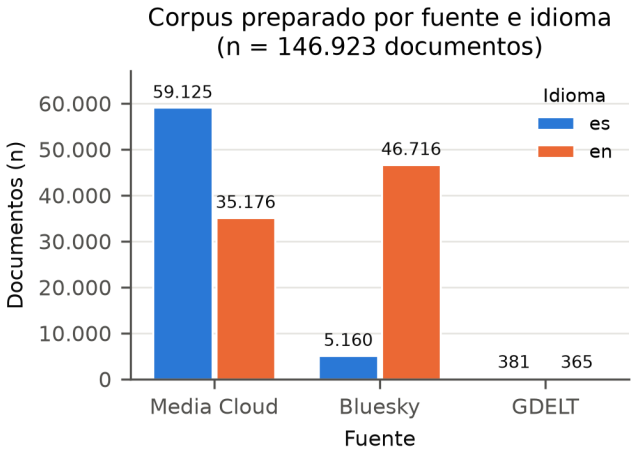


Figura 1. Documentos del corpus preparado (146.923) por fuente e idioma. Las barras de GDELT (381 en español y 365 en inglés) son casi invisibles a esta escala.

III. Segmentación (B)

La segmentación por tokens usa ventanas de 256 tokens con solapamiento de 48 y produce 577.324 chunks en `data/chunks_tok256.parquet`. Al retokenizar, 40.474 chunks superan 256 tokens, con un máximo de 258. Por documento, la mediana es de 3 chunks, el percentil 95 de 10 y el máximo de 426. Estas cifras están verificadas. La Figura 2 muestra la distribución de chunks por documento y la Figura 3, la de tokens por chunk al retokenizar.

IV. Embeddings (C)

El proyecto usa `intfloat/multilingual-e5-small` con el prefijo `passage:` para documentos y `query:` para consultas [6], [7]. Los pesos se descargaron en la caché local de Hugging Face, fijados a la revisión `614241f`.

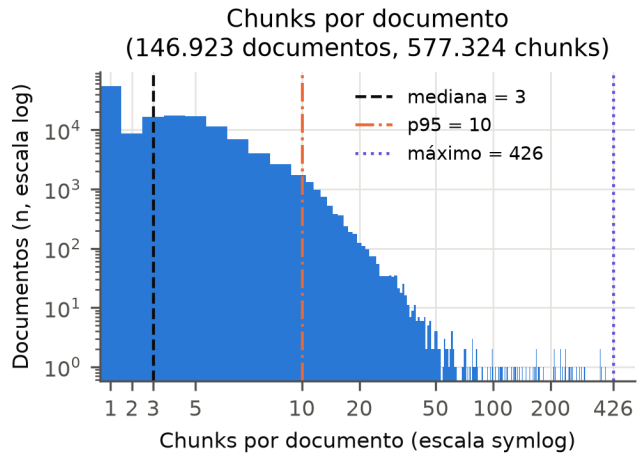


Figura 2. Histograma de chunks por documento (146.923 documentos, 577.324 chunks), con eje vertical logarítmico. Las líneas marcan la mediana (3), el percentil 95 (10) y el máximo (426).

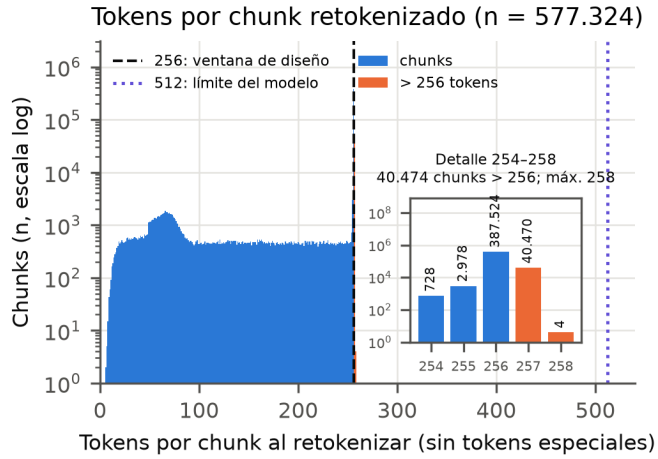


Figura 3. Tokens por chunk al retokenizar los 577.324 chunks sin tokens especiales. Las líneas marcan la ventana de diseño (256) y el límite del modelo (512). El recuadro amplía la cola: 40.474 chunks superan 256 tokens y el máximo es 258.

Se bajaron solo los archivos necesarios para PyTorch y sentence-transformers: unos 493 MB, o 470 MiB en disco. La carga sin conexión con `AutoTokenizer` y `AutoModel` (`local_files_only=True`) funcionó: el modelo tiene una dimensión oculta de 384 y 117.653.760 parámetros.

Los embeddings del piloto ya están calculados: 500 documentos y 1.904 chunks, con el prefijo `passage:`, lotes de 32 y normalización L2, en CPU. El cálculo tardó 22,4 s (11,8 ms por chunk) con un pico de memoria de 1,9 GB. Con la misma tasa, los 577.324 chunks del corpus completo tomarían unas 1,9 h. Son mediciones operativas de un solo equipo, no resultados de recuperación. Después se calcularon los embeddings del corpus completo: 577.324 vectores de dimensión 384, procesados en 29 bloques reanudables de 20.000 chunks. Se verificó que los `chunk_id` son únicos y coinciden con los de la segmentación, que no hay valores NaN y que todas las normas están a menos de 10^{-6} de 1. La comparación con otros encoders sigue pendiente.

V. Milvus (D)

La pila usa Milvus standalone v2.6.25 con etcd embebido y almacenamiento local, sin MinIO. Esta configuración está validada. La colección `rag_e5s_tok256_v1` contiene los 577.324 chunks del corpus completo (Sección VI).

VI. Implementación y configuración congelada

Ingesta por lotes. La primera versión de la ingesta leía todos los vectores a la vez y el proceso llegaba a 9,07 GB de memoria. La versión actual lee el Parquet de embeddings en lotes de 20.000 filas, toma de la segmentación solo los metadatos de esos chunks y hace *upsert* por `chunk_id`. Con ella, la carga completa tardó 115 s y la memoria máxima del proceso bajó a 2,41 GB. Una segunda carga idéntica dejó la colección en las mismas 577.324 filas, contadas con `count(*)`, lo que confirma que la ingesta es idempotente.

Memoria de Milvus. Con el límite inicial de 4 GiB, Milvus se detuvo por falta de memoria al recibir la segunda carga completa, porque el *upsert* conserva las versiones reemplazadas hasta la compactación. Con el límite en 8 GiB las dos cargas terminaron sin reinicios.

API. FastAPI expone `/health`, `/retrieve` y `/ask`. El conteo de filas de `/health` usa `count(*)`, porque las estadísticas de Milvus incluyen las filas reemplazadas hasta la compactación. Cuando Ollama corta una generación por repetición, `/ask` responde con un error específico (HTTP 502) en lugar de uno genérico.

Configuración congelada. Los valores fijos para todas las condiciones están en la sección 9 de `docs/protocolo_evaluacion.md`: `prompt_v1` con su huella sha256, $k = 5$, chunks de 256/48, el encoder E5 en una revisión fija, HNSW con IP, la partición 20/20 y, para la generación, temperatura 0 y `repeat_penalty` 1,1 en los cuatro modelos. La penalización se añadió porque, con temperatura 0 y sin ella, `gemma2:2b` entró en un bucle de repetición con un prompt real y Ollama cortó la generación ("token repeat limit reached"). Con 1,1, `gemma2:2b` responde con citas que apuntan a fragmentos recuperados. El umbral de abstención no forma parte de lo congelado: sigue pendiente de calibración.

Hallazgos de la prueba técnica. En la prueba de `repeat_penalty`, con un solo prompt real y las dos configuraciones, `qwen2.5:1.5b` no citó ningún fragmento y `llama3.2:1b` generó respuestas muy cortas, de 7 a 9 tokens. Son observaciones de una comprobación técnica, no resultados experimentales, pero anticipan diferencias que la evaluación deberá medir.

Estado de la evaluación. El gold standard sigue pendiente de redacción y revisión por el equipo, y sin él no se puede calibrar el umbral de abstención. Los experimentos con los cuatro modelos y la evaluación humana también están pendientes. Las funciones de métricas (Recall@k, MRR, nDCG@k y precisión

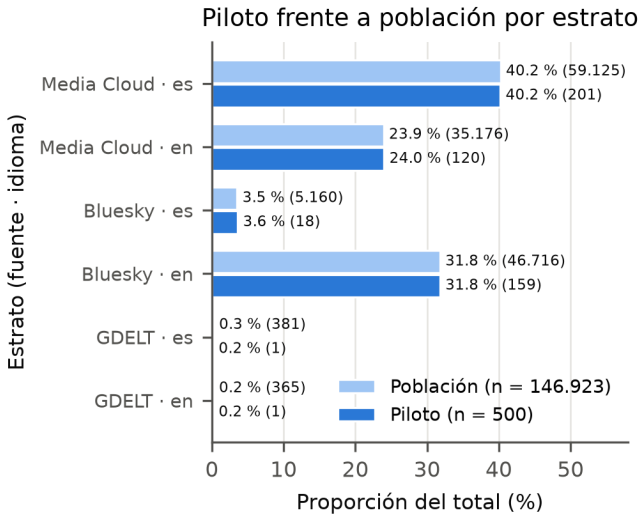


Figura 4. Proporción de cada estrato fuente × idioma en el piloto (500 documentos: 220 en español y 280 en inglés) y en la población (146.923 documentos). Entre paréntesis, los conteos.

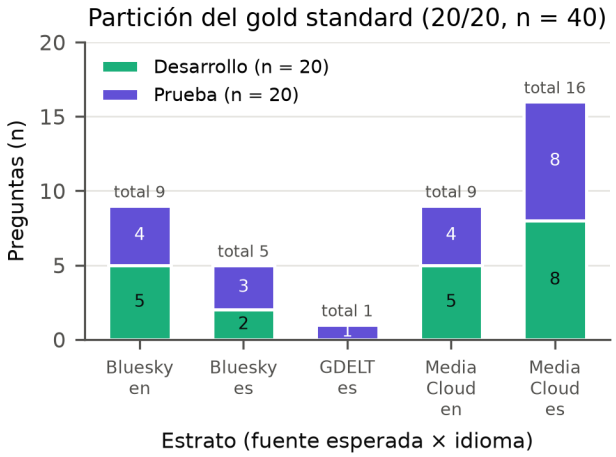


Figura 5. Partición del gold standard por fuente esperada e idioma: 20 preguntas de desarrollo y 20 de prueba (40 en total).

y recall de la abstención) y el validador del gold standard están implementados y probados con datos sintéticos, pero aún no se han aplicado al gold standard.

VII. Diseño experimental (5)

El plan propuesto empieza con un piloto de 500 documentos y escala a 10.000 y luego al corpus completo solo si el hardware lo permite. El piloto se seleccionó con semilla 42, estratificado por fuente (`source`) e idioma (`lang`). La Tabla II solo informa conteos; la Figura 4 compara sus proporciones con las de la población.

Cuadro II
Conteos del piloto de 500 documentos por fuente e idioma.

Fuente	es	en	Total
Bluesky	18	159	177
GDELT	1	1	2
Media Cloud	201	120	321
Total	220	280	500

Entre condiciones experimentales solo cambia el LLM; corpus, segmentación, encoder, índice, `top-k` y prompt se mantienen fijos. Según `docs/EVALUACION.md`, cada condición se repetirá al menos tres veces, registrando el arranque en frío y en caliente. Los modelos locales previstos se servirán con Ollama [28]–[30]. Todavía no se han medido la viabilidad en hardware, el tiempo de indexación ni ninguna métrica. Las referencias de evaluación automática son candidatas por verificar [5], [24], [25].

VIII. Gold standard y partición (6)

El gold standard contiene 40 preguntas: 22 en español y 18 en inglés. Su contenido está pendiente de revisión por el equipo. La Tabla III muestra la distribución por fuente esperada e idioma.

Cuadro III
Distribución del gold standard.

Fuente esperada	es	en	Total
Media Cloud	16	9	25
Bluesky	5	9	14
GDELT	1	0	1
Total	22	18	40

La partición vigente, registrada en `docs/protocolo_evaluacion.md`, divide las 40 preguntas en 20 de desarrollo y 20 de prueba. Usa los IDs de `docs/gold_standard_plantilla.csv`, semilla 42 y estratificación por fuente esperada e idioma de la pregunta, con cuotas asignadas por mayor resto (Tabla IV y Figura 5). El único caso GDELT en español va a prueba, porque un estrato de un solo caso no puede quedar en ambas particiones. Los parámetros se ajustan solo con desarrollo y se congelan antes de evaluar con prueba.

Cuadro IV
Partición desarrollo/prueba del gold standard.

Fuente esperada	Idioma	Desarrollo	Prueba	Total
Bluesky	en	5	4	9
Bluesky	es	2	3	5
GDELT	es	0	1	1
Media Cloud	en	5	4	9
Media Cloud	es	8	8	16
Total		20	20	40

La evaluación humana independiente, la rúbrica y la resolución de desacuerdos están pendientes.

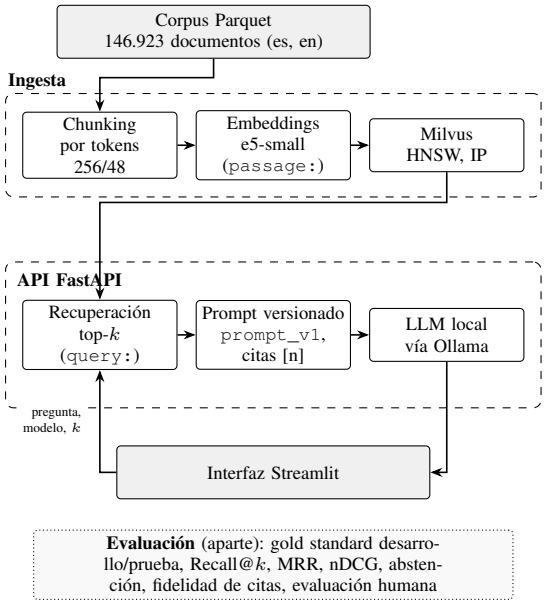


Figura 6. Arquitectura prevista. La ingesta segmenta el corpus, calcula embeddings E5 y los indexa en Milvus; la API recupera fragmentos, arma el prompt versionado y consulta el LLM local; la interfaz solo habla con la API. La evaluación es un bloque aparte que consume las respuestas registradas.

IX. Decisiones de arquitectura

La configuración prevista combina los documentos preparados, la segmentación 256/48, embeddings E5 con prefijo de pasaje, Milvus como almacén vectorial, una API FastAPI, una interfaz Streamlit y LLM locales servidos con Ollama. La Figura 6 muestra el flujo, con la evaluación como bloque aparte.

Encoder. Se usa `multilingual-e5-small`, un encoder multilingüe entrenado con los prefijos `query:` y `passage:` para distinguir consultas de pasajes [7]; el código aplica cada prefijo en la consulta y en la indexación. BGE-M3 es una alternativa multilingüe que además combina varias funciones de recuperación [8]. La comparación controlada entre ambos sigue pendiente, por lo que la elección del encoder todavía no se considera justificada empíricamente.

Índice. Los vectores se normalizan con L2, de modo que el producto interno (IP) equivale al coseno. La colección usa un índice HNSW con métrica IP ($M = 16$, `efConstruction` $= 200$ y `ef` $= 64$ en búsqueda). HNSW organiza los vectores en grafos jerárquicos de mundo pequeño navegables y permite una búsqueda aproximada de vecinos más cercanos eficiente y robusta [10], lo que evita comparar cada consulta con los 577.324 vectores del corpus completo.

Abstención. Un sistema de preguntas y respuestas debe reconocer cuándo la evidencia no permite responder [22], y la abstención es un comportamiento que se estudia de forma específica en los LLM [23]. Self-RAG propone que el modelo decida cuándo recuperar y critique lo recuperado [21]. Además, los modelos de lenguaje aumentados por recuperación deben ser robustos ante contexto irrelevante [19], lo que motiva no entregar al LLM fragmentos con baja puntuación. Aquí se usa una regla más simple: si la mejor puntuación de recuperación queda por debajo de un umbral, el sistema responde que no hay evidencia suficiente y no llama al LLM. El umbral de 0,80 es provisional: las puntuaciones de E5 no se han calibrado en este corpus para decidir la abstención, y el valor se fijará con las 20 preguntas de desarrollo antes de evaluar con prueba.

Contexto. Los LLM aprovechan peor la información situada en el medio de contextos largos [18]. Por eso los fragmentos entran ordenados por puntuación y completos, hasta un presupuesto configurable de tokens (2.048 por defecto); nunca se corta un fragmento. La plantilla `prompt_v1` numera cada fragmento como [n] y pide citarlo, y la API comprueba que cada cita corresponda a un fragmento entregado.

Evaluación de fidelidad. La fidelidad de las respuestas y de las citas se evaluará con anotación humana. RAGAS [24] y ARES [25] proponen marcos automáticos para evaluar sistemas RAG, y el uso de un LLM como juez [26] servirá solo como complemento, contrastado con la evaluación humana.

Inyección de prompt. Los documentos recuperados pueden contener instrucciones que manipulen al LLM (inyección indirecta de prompt) [35], y un atacante podría insertar textos diseñados para alterar las respuestas de un sistema RAG [36]. El prompt ordena ignorar las instrucciones contenidas en los fragmentos; es una mitigación parcial y no se ha evaluado.

Las referencias sobre recuperación híbrida y embeddings son candidatas por verificar [1]–[4], [8], [9].

X. Limitaciones y amenazas a la validez

El gold standard es pequeño y está desequilibrado: Bluesky en español tiene cinco preguntas y GDELT una. Con la partición, la prueba queda con tres preguntas de Bluesky en español y una de GDELT, así que esos estratos tendrán alta incertidumbre. La preparación, los filtros de idioma y la segmentación determinan la población evaluada; cualquier cambio exige documentar una nueva versión de los datos. El umbral de abstención de 0,80 es provisional y no está calibrado; se fijará con las preguntas de desarrollo después de la revisión del gold standard por el equipo. Todas las referencias permanecen por verificar.

XI. Pendientes

- Redacción y revisión del gold standard por el equipo.
- Calibración del umbral de abstención con la partición de desarrollo.
- Verificación de la interfaz Streamlit en ejecución.
- Experimentos: piloto, 10.000 documentos y corpus completo si los recursos lo permiten, con métricas, tiempos, recursos y repeticiones por condición.
- Evaluación humana, medición del acuerdo y resolución de desacuerdos.
- Justificación del encoder frente a alternativas.
- Verificación de cada referencia contra su fuente original.

Referencias

[1] S. Robertson y H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," 2009, doi:10.1561/1500000019. *Por verificar.*

[2] G. V. Cormack, C. L. A. Clarke y S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," 2009, doi:10.1145/1571941.1572114. *Por verificar.*

[3] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," 2020, arXiv:2004.04906. *Por verificar.*

[4] T. Formal, B. Piwowarski y S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," 2021, arXiv:2107.05720. *Por verificar.*

[5] N. Thakur et al., "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," 2021, arXiv:2104.08663. *Por verificar.*

[6] L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training," 2022, arXiv:2212.03533. *Por verificar.*

[7] L. Wang et al., "Multilingual E5 Text Embeddings: A Technical Report," 2024, arXiv:2402.05672. *Por verificar.*

[8] J. Chen et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024, arXiv:2402.03216. *Por verificar.*

[9] N. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," 2022, arXiv:2210.07316. *Por verificar.*

[10] Y. Malkov y D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE TPAMI*, 2018. *Por verificar.*

[11] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," 2023, arXiv:2210.09984. *Por verificar.*

[12] R. Nogueira y K. Cho, "Passage Re-ranking with BERT," 2019, arXiv:1901.04085. *Por verificar.*

[13] O. Khattab y M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020, arXiv:2004.12832. *Por verificar.*

[14] W. Sun et al., "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents," 2023, arXiv:2304.09542. *Por verificar.*

[15] Anthropic, "Introducing Contextual Retrieval," 2024, https://www.anthropic.com/news/contextual-retrieval. *Por verificar.*

[16] M. Günther et al., "Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models," 2024, arXiv:2409.04701. *Por verificar.*

[17] P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," 2024, arXiv:2401.18059. *Por verificar.*

[18] N. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *TACL*, 2024. *Por verificar.*

[19] O. Yoran et al., "Making Retrieval-Augmented Language Models Robust to Irrelevant Context," 2023, arXiv:2310.01558. *Por verificar.*

[20] S.-Q. Yan et al., "Corrective Retrieval Augmented Generation (CRAG)," 2024, arXiv:2401.15884. *Por verificar.*

[21] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," 2023, arXiv:2310.11511. *Por verificar.*

[22] P. Rajpurkar, R. Jia y P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD," 2018, arXiv:1806.03822. *Por verificar.*

[23] B. Wen et al., "Know Your Limits: A Survey of Abstention in Large Language Models," 2024, arXiv:2407.18418. *Por verificar.*

[24] S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation," 2023, arXiv:2309.15217. *Por verificar.*

[25] J. Saad-Falcon et al., "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," 2023, arXiv:2311.09476. *Por verificar.*

[26] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023, arXiv:2306.05685. *Por verificar.*

[27] K. Järvelin y J. Kekäläinen, "Cumulated Gain-Based Evaluation of IR Techniques," 2002, doi:10.1145/582415.582418. *Por verificar.*

[28] Qwen Team, "Qwen2.5 Technical Report," 2024, arXiv:2412.15115. *Por verificar.*

[29] Gemma Team, "Gemma 2: Improving Open Language Models at a Practical Size," 2024, arXiv:2408.00118. *Por verificar.*

[30] A. Grattafiori et al. (Llama Team), "The Llama 3 Herd of Models," 2024, arXiv:2407.21783. *Por verificar.*

[31] M. Abdin et al., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," 2024, arXiv:2404.14219. *Por verificar.*

[32] A. Z. Broder, "On the Resemblance and Containment of Documents," 1997, doi:10.1109/SEQUEN.1997.666900. *Por verificar.*

[33] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002, doi:10.1145/509907.509965. *Por verificar.*

[34] J. Leskovec, L. Backstrom y J. Kleinberg, "Meme-tracking and the Dynamics of the News Cycle," 2009, doi:10.1145/1557019.1557077. *Por verificar.*

[35] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," 2023, arXiv:2302.12173. *Por verificar.*

[36] W. Zou et al., "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models," 2024, arXiv:2402.07867. *Por verificar.*